📋 Mémo Rsyslog, journalisation & RainerScript

Pour une étudiante en 2e année BTS SIO – option SISR

---

1. La journalisation, c'est quoi ?

Journaliser = enregistrer des événements (connexions, erreurs, alertes…) dans des **fichiers de logs**.

Pourquoi c'est vital en SISR :

- 🔍 **Diagnostic** : comprendre pourquoi un service plante
- 🛡️ **Sécurité** : détecter des intrusions, tracer les connexions
- 📊 **Supervision** : suivre l'état des machines
- ⚖️ **Conformité** : preuve légale, audit (RGPD, ISO 27001…)

> 💡 Sans logs, débugger un serveur, c'est jouer à la devinette.

---

2. Syslog : la base

Syslog = protocole standard (RFC 5424) qui définit le **format** des messages et leur **transport** (UDP 514 par défaut, TCP 514 ou 6514 en TLS pour la fiabilité).

Format d'un message syslog

```
PRI TIMESTAMP HOSTNAME PROCESS PID MESSAGE
```

Exemple :

```
<34>Feb 12 10:30:00 srv-web sshd[1234]: Failed password for root
```

Gravité (severity) — 8 niveaux, du plus grave au moins grave

Code	Niveau	Mots-clés	Usage
0	emerg	`panic`	Système inutilisable 🚨
1	alert		Action immédiate requise
2	crit		État critique
3	err	`error`	Erreur
4	warning	`warn`	Avertissement
5	notice		Normal mais notable
6	info		Information
7	debug		Débogage (bavard !)

> 🧠 Astuce mémo : **"Every Awesome Cisco Engineer Will Need Ice-cream Daily"** (Emergency → Debug)

Facilities (qui parle ?)

`auth` (connexion), `authpriv`, `cron`, `daemon`, `kern` (noyau), `mail`, `syslog`, `user`, `local0` à `local7` (personnalisables — souvent utilisées pour les équipements réseau).

---

3. Rsyslog : le syslog "amélioré"

Rsyslog = implémentation moderne de syslog, **démon par défaut** sur Debian/Ubuntu/RHEL. Multi-threadé, protocole **RainerScript** (voir §5), modules d'entrée/sortie, filtres puissants.

Fichiers importants

Fichier	Rôle
`/etc/rsyslog.conf`	Fichier principal de configuration
`/etc/rsyslog.d/*.conf`	Configs additionnelles (incluses via `$IncludeConfig`)
`/var/log/syslog` (Debian) ou `/var/log/messages` (RHEL)	Log général
`/var/log/auth.log`	Authentifications, sudo, SSH
`/var/log/kern.log`	Messages du noyau
`/var/log/cron`	Tâches planifiées

Commandes à connaître

```
# Redémarrer après modification de la config
systemctl restart rsyslog
# Tester la syntaxe de la config
rsyslogd -N1
# Voir les logs en direct
tail -f /var/log/syslog
# Envoyer un message de test manuellement
logger -p local0.warning "Test depuis le mémo !"
```

Configuration classique (legacy format)

Syntaxe : `FACILITY.SEVERITY ACTION`

```
# Tout ce qui est info et plus grave va dans syslog
*.info;mail.none;authpriv.none /var/log/syslog
# Les messages d'authentification
auth,authpriv.* /var/log/auth.log
# Sauver les messages critiques partout
*.=crit /var/log/warnings
```

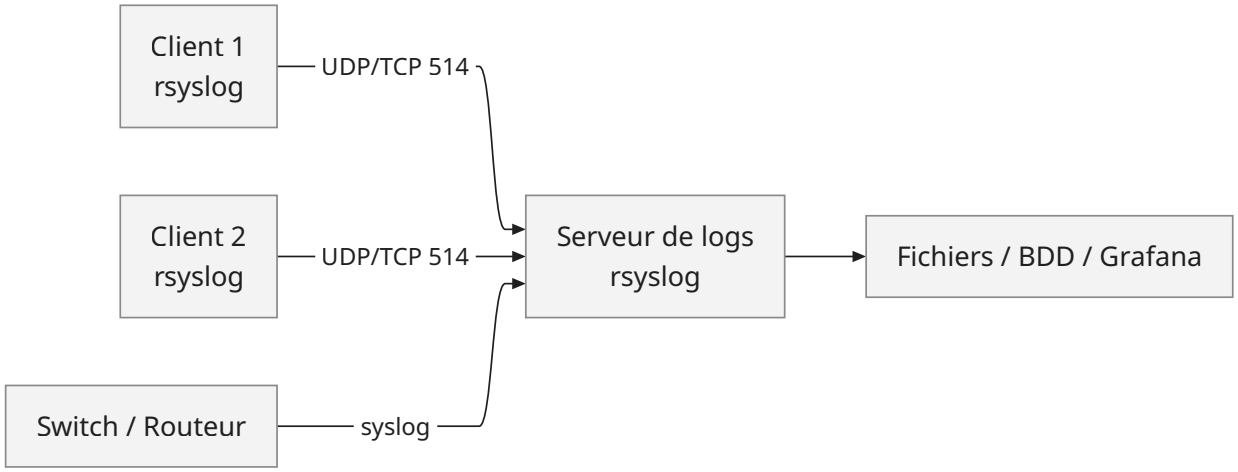
Sélecteurs à maîtriser :

- `*` = tout ; `.info` = info **et plus grave** ; `.=debug` = **uniquement** debug ; `.!=err` = tout **sauf** err
- `mail.none` = exclure la facility mail
- Plusieurs sélecteurs séparés par `;`

---

4. Centralisation des logs (grand classique du SISR !)

Schéma typique : les clients envoient leurs logs à un **serveur de logs central**.



Côté client (émetteur)

```
# dans /etc/rsyslog.d/50-remote.conf
*.* @192.168.1.100:514 # @ = UDP
*.* @@192.168.1.100:514 # @@ = TCP
```

Côté serveur (récepteur)

```
# Activer la réception (modules déjà présents, juste décommenter)
module(load="imudp")
input(type="imudp" port="514")
# ou en TCP :
module(load="imtcp")
input(type="imtcp" port="514")
# Trier les logs par machine émettrice
$template RemoteHost,"/var/log/remote/%HOSTNAME%.log"
*.* ?RemoteHost
```

> 🛡️ Sécurité : privilégier **TCP ou TLS (port 6514)** plutôt qu'UDP, qui ne garantit ni la livraison ni l'ordre. Et **penser au pare-feu** : ouvrir le port 514 !

5. RainerScript : la syntaxe moderne

C'est le **langage de script de rsyslog** (créé par Rainer Gerhards, auteur de rsyslog). Plus lisible et puissant que le format legacy. Les deux formats cohabitent dans le même fichier.

a) Structure de base : ruleset, action, template

```
# Définir un template (format de fichier personnalisé)
template(name="monformat" type="string"
 string="/var/log/%HOSTNAME%/%PROGRAMNAME%.log")
# Une règle : condition => action
if $programname == 'sshd' then {
 action(type="omfile" file="/var/log/ssh.log")
}
```

b) Conditions

```
if $syslogseverity <= 3 then action(type="omfile" file="/var/log/erreurs.log")
if $hostname startswith 'web' and $syslogfacility-text == 'auth' then {
 action(type="omfwd" target="10.0.0.5" port="514" protocol="tcp")
}
if ($msg contains 'Failed password') then {
 action(type="omfile" file="/var/log/bruteforce.log")
}
```

Propriétés utiles : `$hostname`, `$programname`, `$msg`, `$syslogseverity`, `$syslogfacility-text`, `$timereported`, `$fromhost-ip`.

c) Les actions principales

Action	Rôle	Exemple clé
`omfile`	Écrire dans un fichier	`file="/var/log/x.log"`
`omfwd`	Transférer à un autre serveur	`target="IP" port="514" protocol="tcp"`
`ommysql`	Écrire dans MySQL	`server="localhost" db="Syslog"`
`discard`	Jeter le message	`stop` (mot-clé)

```
# Transfert réseau moderne
action(type="omfwd" target="192.168.1.100" port="514" protocol="tcp"
 queue.type="LinkedList" queue.size="10000")
# Arrêter le traitement (le message n'est plus traité ensuite)
if $programname == 'systemd' then stop
```

d) Templates et format des messages

```
template(name="detailed" type="list") {
 property(name="timereported" dateFormat="rfc3339")
 constant(value=" ")
 property(name="hostname")
 constant(value=" ")
 property(name="msg" sp-if-no-1st-sp="1" format="json")
}
```

> 💡 `type="list"` est la forme recommandée ; `type="string"` est plus simple pour débuter.

e) Exemple complet type examen

```
# /etc/rsyslog.d/90-monlab.conf
# 1. Les erreurs SSH dans un fichier dédié
if $programname == 'sshd' and $syslogseverity <= 4 then {
 action(type="omfile" file="/var/log/ssh-alertes.log")
}
# 2. Tout transférer au serveur central en TCP
action(type="omfwd" target="192.168.1.100" port="514" protocol="tcp")
# 3. Ignorer le bruit de systemd
if $programname == 'systemd' then stop
```

---

6. Logrotate : la rotation des logs 🔄

Les logs grossissent à l'infini → il faut les **rotater** (archiver, compresser, purger). Sur Debian : outil **logrotate**, config dans `/etc/logrotate.conf` et `/etc/logrotate.d/`.

```
/var/log/syslog {
 daily # rotation quotidienne
 rotate 7 # garder 7 fichiers
 compress # compresser les anciens
 missingok # pas d'erreur si absent
 notifempty # ignorer les fichiers vides
 create 640 syslog adm
 postrotate
 systemctl reload rsyslog >/dev/null 2>&1 || true
 endscript
}
```

> 🧠 À retenir : **rotation** = quand on archive ; **rétention** = combien de temps on garde.

---

7. Fichiers de logs à connaître par cœur 📚

Fichier	Contenu
`/var/log/syslog` / `messages`	Journal général
`/var/log/auth.log` / `secure`	Connexions, sudo, SSH
`/var/log/kern.log`	Noyau
`/var/log/dmesg`	Démarrage / matériel
`/var/log/cron`	Tâches planifiées
`/var/log/mail.log`	Serveur de messagerie
`/var/log/faillog`, `lastlog`, `wtmp`	Échecs / dernières connexions (`lastlog`, `last`)
`/var/log/journal/`	Journaux de **systemd-journald** (binaire)

Et côté **journald** : `journalctl -u ssh`, `journalctl -f`, `journalctl --since "1 hour ago"`. Journald et rsyslog cohabitent : journald collecte, rsyslog filtre/centralise.

---

8. Checklist de révision ✅

- [ ] Citer les 8 niveaux de gravité et 3-4 facilities
- [ ] Expliquer la différence UDP (`@`) / TCP (`@@`) / TLS (`@@` + port 6514)
- [ ] Écrire une règle legacy `facility.severity fichier`
- [ ] Écrire une règle RainerScript `if ... then action(...)`
- [ ] Créer un template et l'utiliser dans une action
- [ ] Configurer un client et un serveur de logs centralisés
- [ ] Envoyer un test avec `logger` et vérifier avec `tail -f`
- [ ] Valider la config avec `rsyslogd -N1` et redémarrer le service
- [ ] Expliquer logrotate (rotation + rétention)
- [ ] Savoir où chercher : `auth.log`, `syslog`, `journalctl`

---

9. Pièges classiques ⚠️

1. **Oublier de redémarrer rsyslog** après un changement de config → rien ne bouge !
2. **UDP = pas de garantie** : un log "perdu" sur le réseau n'existe jamais côté serveur.
3. `stop` coupe le traitement : placer les règles de filtrage **dans le bon ordre**.
4. Confondre **facility** (qui parle) et **severity** (à quel point c'est grave).
5. Un pare-feu qui bloque le 514 : la centralisation "marche pas" 9 fois sur 10 à cause de ça.
6. `.info` signifie "info et plus grave", pas "uniquement info" — piège d'examen favori 😈

Bonne révision ! 🍀 En cas de doute en TP : `rsyslogd -N1`, `systemctl restart rsyslog`, *puis* `logger "test"` *et* `tail -f`.